NAME : YUGITHA B

REG NO : 192424102

Part A: Data Types in R

1. **Define data types in R. Explain the different basic data types with suitable examples.**

Data Type	Description	Example
Integer	Whole numbers	25L
Numeric	Decimal numbers	25.5
Character	Text or strings	"Hello"
Logical	TRUE or FALSE values	TRUE
Complex	Complex numbers	3 + 4i

2. **Create variables of different data types and identify their types using** class()**.**

```
a <- 25L
b <- 25.5
c <- "R Programming"
d <- TRUE
e <- 3 + 4i
print(a)
print(class(a))
print(b)
print(class(b))
print(c)
print(class(c))
print(d)
print(class(d))
print(e)
print(class(e))
```

OUTPUT

```
> print(a)
[1] 25
> print(class(a))
[1] "integer"
>
> print(b)
[1] 25.5
> print(class(b))
[1] "numeric"
>
> print(c)
[1] "R Programming"
> print(class(c))
[1] "character"
>
> print(d)
[1] TRUE
> print(class(d))
[1] "logical"
>
> print(e)
[1] 3+4i
> print(class(e))
[1] "complex"
>
```

3. **Differentiate between integer and numeric data types in R.**

Integer	Numeric
Stores whole numbers	Usually stores decimal numbers
Written with L	Usually written without L
Example: 25L	Example: 25.5
class(25L) gives "integer"	class(25.5) gives "numeric"

4. Explain typeof()**,** class()**, and** is.numeric()**.**

typeof()

x <- 25.5

typeof(x)

```
> x <- 25.5
> typeof(x)
[1] "double"
>
```

class()

x <- 25.5

class(x)

```
> x <- 25.5
> class(x)
[1] "numeric"
>
```

is.numeric()

x <- 25.5

is.numeric(x)

```
> x <- 25.5
> is.numeric(x)
[1] TRUE
```

5. Convert numeric to character and character to numeric.

Numeric to character

x <- 25.5

y <- as.character(x)

print(y)

print(class(y))

Character to numeric

a <- "100"

b <- as.numeric(a)

print(b)

print(class(b))

```
> # Numeric to character
> x <- 25.5
> y <- as.character(x)
>
> print(y)
[1] "25.5"
> print(class(y))
[1] "character"
>
> # Character to numeric
> a <- "100"
> b <- as.numeric(a)
>
> print(b)
[1] 100
> print(class(b))
[1] "numeric"
```

6. Identify the data type of each variable.

Variable	Value	Data Type	Justification
a	25L	Integer	L specifies an integer
b	25.5	Numeric	Decimal value
c	"R Programming"	Character	Text is enclosed in quotes
d	FALSE	Logical	TRUE/FALSE value
e	3 + 4i	Complex	Contains imaginary part i

7. Explain type coercion in R.

Type coercion is the automatic conversion of one data type into another data type.

x <- c(10, "20", TRUE)

print(x)

```
[1] "numeric"
> x <- c(10, "20", TRUE)
> print(x)
[1] "10"   "20"   "TRUE"
```

Part B: Vectors in R

8. Define a vector in R. Explain how to create different vectors using c().

A vector is a collection of elements of the same basic data type.

The c() function is used to combine values and create a vector.

Numeric vector:

x <- c(10, 20, 30, 40)

Character vector:

names <- c("Ravi", "Anu", "Kumar")

Logical vector:

result <- c(TRUE, FALSE, TRUE)

9. Create a numeric vector containing ten elements and perform arithmetic operations.

x <- c(10, 20, 30, 40, 50, 60, 70, 80, 90, 100)

Addition

print(x + 5)

Subtraction

print(x - 5)

Multiplication

print(x * 2)

Division

```
> # Addition
> print(x + 5)
 [1]  15  25  35  45  55  65  75  85  95 105
>
> # Subtraction
> print(x - 5)
 [1]  5 15 25 35 45 55 65 75 85 95
>
> # Multiplication
> print(x * 2)
 [1]  20  40  60  80 100 120 140 160 180 200
>
> # Division
> print(x / 2)
 [1]  5 10 15 20 25 30 35 40 45 50
```

print(x / 2)

10. Explain vector indexing in R.

Vector indexing is used to access specific elements of a vector.

Consider:

R

x <- c(10, 20, 30, 40, 50)

Access individual element:

R

x[2]

Output:

20

Select multiple elements:

R

x[c(1, 3, 5)]

Output:

10 30 50

Exclude elements using negative indices:

R

x[-2]

Output:

10 30 40 50

R uses **1-based indexing**, so the first element is at position 1.

11. Create a vector of student marks and perform the operations.

marks <- c(80, 65, 90, 75, 88, 70, 95)

Sum

print(sum(marks))

Average

print(mean(marks))

Maximum

print(max(marks))

Minimum

print(min(marks))

Marks greater than 75

print(marks[marks > 75])

```
 [1]  5 10 15 20 25 30 35 40 45 50
> marks <- c(80, 65, 90, 75, 88, 70, 95)
>
> # Sum
> print(sum(marks))
[1] 563
>
> # Average
> print(mean(marks))
[1] 80.42857
>
> # Maximum
> print(max(marks))
[1] 95
>
> # Minimum
> print(min(marks))
[1] 65
>
> # Marks greater than 75
> print(marks[marks > 75])
[1] 80 90 88 95
```

12. Explain vector recycling in R.

Vector recycling means R repeats the shorter vector when performing an operation with a longer vector.

Example:

R

x <- c(10, 20, 30, 40)

y <- c(2, 4)

x + y

R repeats y as:

2, 4, 2, 4

So:

10+2 = 12

20+4 = 24

30+2 = 32

40+4 = 44

Output:

[1] 12 24 32 44

13. Create a character vector of five student names.

names <- c("Arun", "Bala", "Divya", "Kavi", "Meena")

Sort alphabetically

print(sort(names))

Find length

print(length(names))

Access third element

print(names[3])

```
[1] 80 90 88 95
> names <- c("Arun", "Bala", "Divya", "Kavi", "Meena")
>
> # Sort alphabetically
> print(sort(names))
[1] "Arun"  "Bala"  "Divya" "Kavi"  "Meena"
>
> # Find length
> print(length(names))
[1] 5
>
> # Access third element
> print(names[3])
[1] "Divya"
```

14. Determine the output of each operation.

a) x + y

x <- c(10, 20, 30, 40, 50)

y <- c(2, 4, 6, 8, 10)

x + y

```
> x <- c(10, 20, 30, 40, 50)
> y <- c(2, 4, 6, 8, 10)
> x + y
[1] 12 24 36 48 60
```

b) x * 2

x <- c(10, 20, 30, 40, 50)

y <- c(2, 4, 6, 8, 10)

x * 2

```
[1] 12 24 36 48 60
> x <- c(10, 20, 30, 40, 50)
> y <- c(2, 4, 6, 8, 10)
> x * 2
[1]  20  40  60  80 100
```

c) x[c(1, 3, 5)]

x <- c(10, 20, 30, 40, 50)

y <- c(2, 4, 6, 8, 10)

x[c(1, 3, 5)]

```
[1]  20  40  60  80 100
> x <- c(10, 20, 30, 40, 50)
> y <- c(2, 4, 6, 8, 10)
> x[c(1, 3, 5)]
[1] 10 30 50
```

15. College attendance program

attendance <- c(90, 82, 88, 75, 95, 86)

Students with attendance above 85%

print(attendance[attendance > 85])

Average attendance

print(mean(attendance))

Number of students meeting requirement

print(sum(attendance > 85))

```
[1] 10 30 50
> attendance <- c(90, 82, 88, 75, 95, 86)
>
> # Students with attendance above 85%
> print(attendance[attendance > 85])
[1] 90 88 95 86
>
> # Average attendance
> print(mean(attendance))
[1] 86
>
> # Number of students meeting requirement
> print(sum(attendance > 85))
[1] 4
```